Елисеева Е.Р. 090303-ПИа-о24

Лабораторная работа №2
Коллекции, строки и функции в Python
Вариант №5. Фильмы

1. Цель работы

Получить практические навыки работы с основными структурами данных языка Python (списки, словари, множества), научиться применять их для обработки наборов информации, реализовывать фильтрацию, поиск и сортировку элементов с использованием встроенных инструментов и списковых включений (list comprehensions), а также структурировать код с помощью пользовательских функций и организовывать интерактивное меню управления приложением.

2. Постановка индивидуального задания

Разработать консольное приложение для обработки информации о фильмах. Для каждого фильма необходимо хранить следующие параметры: название, жанр, год выпуска и рейтинг. Программа должна быть структурирована с помощью пользовательских функций, хранить данные в списке словарей и обеспечивать выполнение следующих обязательных операций через интерактивное меню на базе цикла while:

1) Добавление нового фильма с помощью пользовательского ввода (год выпуска от 1895 до 2026, рейтинг от 0 до 10);

2) Вывод на экран полной информации обо всех зарегистрированных фильмах;

3) Поиск фильмов по заданному жанру (без учета регистра букв);

4) Фильтрация фильмов по минимально заданному рейтингу;

5) Сортировка всей коллекции фильмов по убыванию их рейтинга;

6)Расчет и вывод не менее трех статистических показателей (общее количество, средний рейтинг, фильм с максимальным рейтингом);

7)Определение и отображение множества уникальных жанров фильмов в коллекции.

3. Краткие теоретические сведения

Коллекция в Python — это структура данных, предназначенная для группировки и хранения нескольких элементов как единого целого. К основным встроенным типам относятся изменяемые упорядоченные списки (list), неизменяемые кортежи (tuple), множества уникальных значений (set) и словари (dict), хранящие пары «ключ-значение».

Список словарей — это комбинированная структура данных (массив объектов), которая идеально подходит для моделирования реальных объектов и сущностей. Главный список выступает в роли таблицы или базы данных, а каждый вложенный словарь представляет собой отдельную запись со своими именованными атрибутами. Данная структура широко применяется при обмене данными через JSON и REST API.

Списковые включения (list comprehensions) предоставляют компактный и синтаксически чистый способ генерации и фильтрации списков на основе существующих коллекций всего в одну строку кода. Они работают быстрее стандартных циклов с явным вызовом метода append() за счет внутренней оптимизации интерпретатора. Синтаксис включает выражение результата, итератор по коллекции и необязательное логическое условие.

Множество (set) — это неупорядоченная коллекция, которая по определению содержит только уникальные элементы. Передача списка в конструктор set() автоматически удаляет все дубликаты, что крайне эффективно для выделения уникальных категорий или параметров из общего массива данных.

4. Исходный код программы

```
def add_movie(movies):
    print("\nдобавление нового фильма:")
    name_i = input('введите название фильма: ').strip()
    genre_i = input('введите жанр фильма: ').strip()
    while True:
        try:
            year_i = int(input('введите год выпуска: '))
            if year_i < 1895 or year_i > 2026:
                print('в этом году нет фильмов :(\nпопробуйте ввести другой
год:')
                continue
            break
        except ValueError:
            print('введите год целым числом')
    while True:
        try:
            rating_i = float(input('введите рейтинг: '))
            if rating_i > 10 or rating_i < 0:
                print('рейтинг должен быть от 0 до 10')
                continue
            break
        except ValueError:
            print('введите число (можно дробное через точку), а не текст')
    movie_dict = {
        'name': name_i,
        'genre': genre_i,
        'year': year_i,
        'rating': rating_i
    }
    movies.append(movie_dict)
    print(f"фильм '{name_i}' успешно добавлен!")


def show_movies(movies):
    if not movies:
        print("\nсписок фильмов пока пуст :(")
        return
    print("\nсписок всех фильмов:")
    for idx, movie in enumerate(movies, 1):
```

```
        print(f"{idx}. {movie['name']} | Жанр: {movie['genre']} | Год:
{movie['year']} | Рейтинг: {movie['rating']}")


def search_by_genre(movies, search_genre):
    return [m for m in movies if m['genre'].lower() == search_genre.lower()]


def filter_by_rating(movies, min_rating):
    return [m for m in movies if m['rating'] >= min_rating]


def sort_movies_by_rating(movies):
    return sorted(movies, key=lambda x: x['rating'], reverse=True)


def show_statistics(movies):
    if not movies:
        print("\nнет данных для расчета статистики")
        return
    total = len(movies)
    all_ratings = [m['rating'] for m in movies]
    avg_rating = round(sum(all_ratings) / total, 2)
    max_movie = max(movies, key=lambda x: x['rating'])

    print("\nстатистические показатели:")
    print(f"1) общее количество фильмов: {total}")
    print(f"2) средний рейтинг фильмов: {avg_rating}")
    print(f"3) фильм с максимальным рейтингом: '{max_movie['name']}'
({max_movie['rating']})")


def show_unique_genres(movies):
    if not movies:
        print("\nсписок фильмов пуст")
        return
    genres_set = set([m['genre'].strip().capitalize() for m in movies])
    print("\nмножество уникальных жанров:")
    for genre in genres_set:
        print(f"- {genre}")
```

```
def main():
    movies = []
    while True:
        print("\n===== МЕНЮ =====")
        print("1. добавить фильм")
        print("2. показать все фильмы")
        print("3. найти фильмы по жанру")
        print("4. фильтрация по рейтингу")
        print("5. сортировка по рейтингу")
        print("6. показать статистику")
        print("7. показать уникальные жанры")
        print("0. выход")
        choice = input("выберите действие: ").strip()
        if choice == "1":
            add_movie(movies)
        elif choice == "2":
            show_movies(movies)
        elif choice == "3":
            g = input("введите жанр для поиска: ").strip()
            found = search_by_genre(movies, g)
            if found:
                print(f"\nнайденные фильмы в жанре '{g}':")
                for m in found:
                    print(f"- {m['name']} ({m['year']}) — рейтинг:
{m['rating']}")
            else:
                print(f"\nфильмы в жанре '{g}' не найдены")
        elif choice == "4":
            try:
                r = float(input("показать фильмы с рейтингом не ниже: "))
                filtered = filter_by_rating(movies, r)
                if filtered:
                    print(f"\nфильмы с рейтингом >= {r}:")
                    for m in filtered:
                        print(f"- {m['name']} рейтинг: {m['rating']} жанр:
{m['genre']}")
                else:
                    print(f"\nфильмы с рейтингом {r} и выше отсутствуют")
            except ValueError:
                print("ошибка: рейтинг должен быть числом")
```

```
        elif choice == "5":
            sorted_list = sort_movies_by_rating(movies)
            print("\nфильмы, отсортированные по убыванию рейтинга")
            for m in sorted_list:
                print(f"[{m['rating']}] {m['name']} ({m['year']}) —
{m['genre']}")
        elif choice == "6":
            show_statistics(movies)
        elif choice == "7":
            show_unique_genres(movies)
        elif choice == "0":
            print("работа программы завершена")
            break
        else:
            print("неверный пункт меню, выберите действие от 0 до 7")
```

5. Результаты тестирования программы

Тест 1: Обычный набор данных (добавление и вывод элементов)

Описание: Ввод последовательности из 5 разных фильмов для наполнения базы данных и последующий вызов функции просмотра всех записей.

Входные данные:

Фильмы:

1. 'Интерстеллар', Фантастика, 2014, 8.6
2. 'Начало', Фантастика, 2010, 8.5
3. 'Зеленая миля', Драма, 1999, 9.1
4. 'Леон', Боевик, 1994, 8.7
5. 'Матрица', Фантастика, 1999, 8.5

Ожидаемый результат:

Программа успешно сохраняет каждый объект в виде словаря. При выборе пункта 2 выводится таблица со всеми 5 элементами. Это показано на рисунке 2.1, который приведен ниже.

```
выберите действие: 2

список всех фильмов:
1. интерстеллар | Жанр: фантастика | Год: 2014 | Рейтинг: 8.4
2. начало | Жанр: фантастика | Год: 2010 | Рейтинг: 8.5
3. зеленая миля | Жанр: драма | Год: 1999 | Рейтинг: 9.1
4. леон | Жанр: боевик | Год: 1994 | Рейтинг: 8.7
5. матрица | Жанр: фантастика | Год: 1999 | Рейтинг: 8.5

===== МЕНЮ =====
1. добавить фильм
2. показать все фильмы
3. найти фильмы по жанру
4. фильтрация по рейтингу
5. сортировка по рейтингу
6. показать статистику
7. показать уникальные жанры
```

Рисунок 2.1 - успешное сохранение словаря

Тест 2: Набор данных с одинаковыми значениями параметров (сортировка)

Описание: Тестирование работы алгоритма сортировки по рейтингу и извлечения уникальных жанров, когда у нескольких фильмов совпадают параметры рейтинга и жанров.

Входные данные те же, что и в тесте 1.

Вызов пункта 5 (Сортировка по рейтингу) и пункта 7 (Уникальные жанры).

Ожидаемый результат:

Фильмы с одинаковым рейтингом 8.5 ('Начало' и 'Матрица') выводятся корректно друг за другом, продемонстрировано на рисунке 2.2. Множество уникальных жанров выводит без повторений ровно три строки: Фантастика, Драма, Боевик (см. Рисунок 2.3).

```
0. выход
выберите действие: 5

фильмы, отсортированные по убыванию рейтинга
[9.1] зеленая миля (1999) — драма
[8.7] леон (1994) — боевик
[8.5] начало (2010) — фантастика
[8.5] матрица (1999) — фантастика
[8.4] интерстеллар (2014) — фантастика
```

Рисунок 2.2 - сортировка по рейтингу

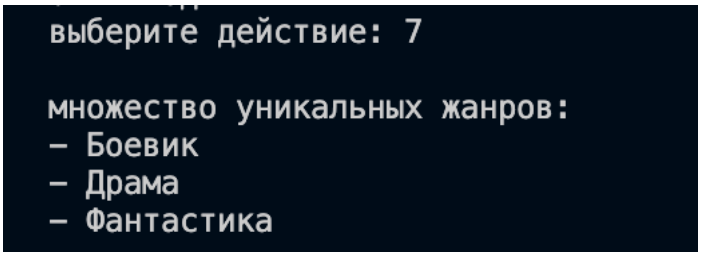


Рисунок 2.3 - уникальные жанры

Тест 3: Проверка программы на обработку допустимых значений года выпуска (1895 и 2026) и рейтинга (0.0 и 10.0), а также попытка ввода некорректных типов данных.

Входные данные:

Попытка ввести год 'привет' (строка) - ошибка, ожидание повторного ввода.

Попытка ввести год 1800 - ошибка.

Ввод граничных значений: Фильм 'Прибытие поезда', Документальный, Год: 1895, Рейтинг: 10.0

Ожидаемый результат:

Конструкция try-except успешно распознала текстовую ошибку. Год 1800 был отклонен условным оператором. Запись с годом 1895 и рейтингом 10.0 успешно добавлена в список.

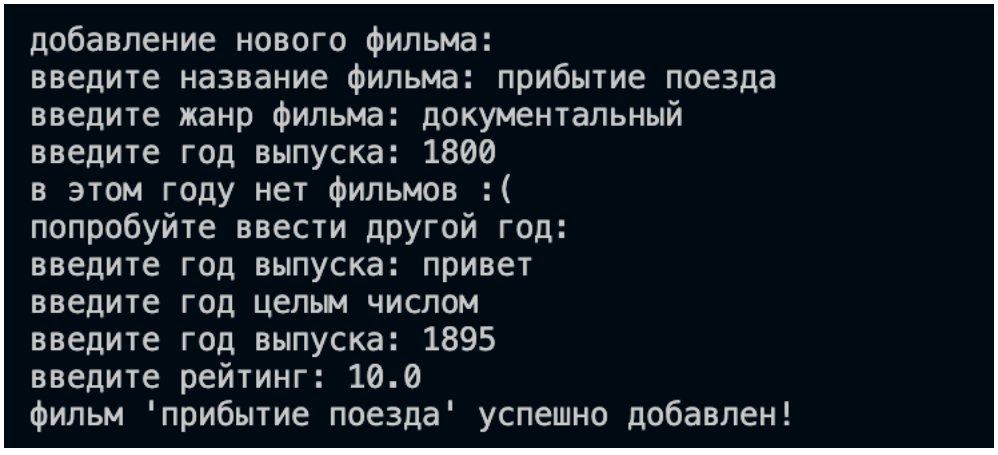


Рисунок 2.4 - ошибки

Тест 4: Поиск параметра, при котором подходящие объекты отсутствуют

Описание: Проверка корректной отработки функции search_by_genre при вводе жанра, которого заведомо нет в сформированной коллекции.

Входные данные:

Выбор пункта 3. Ввод жанра для поиска: 'Ужасы'

Ожидаемый результат:

Программа не генерирует системных ошибок, срабатывает блок else и выводит сообщение: 'фильмы в жанре "ужасы" не найдены' , приведено на рисунке 2.5.

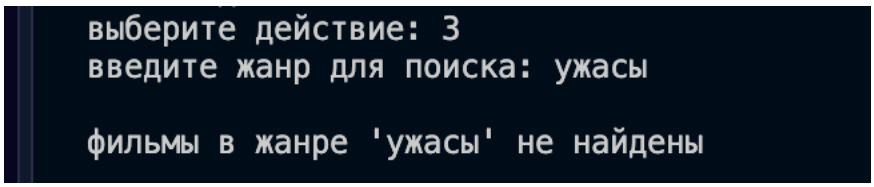


Рисунок 2.5 - поиск отсутствующего параметра

6. Вывод

В ходе выполнения лабораторной работы были успешно освоены базовые и комплексные структуры данных языка Python. На практике изучены механизмы построения комбинированных коллекций типа «список словарей», где список выполняет роль динамической базы данных, а словари хранят именованные свойства конкретных объектов (фильмов). Реализована надежная многоуровневая валидация пользовательского ввода числовых данных (int, float) с помощью обработки исключений try-except и циклов while True, что предотвращает аварийное завершение программы при вводе некорректных типов данных. Были успешно решены задачи фильтрации и поиска элементов с использованием продвинутого синтаксиса списковых включений (list comprehensions), обеспечивающих высокую читаемость кода и оптимизацию производительности. Программа спроектирована по модульному принципу в

виде отдельных независимых функций, управление которыми централизовано через интерактивное консольное меню.